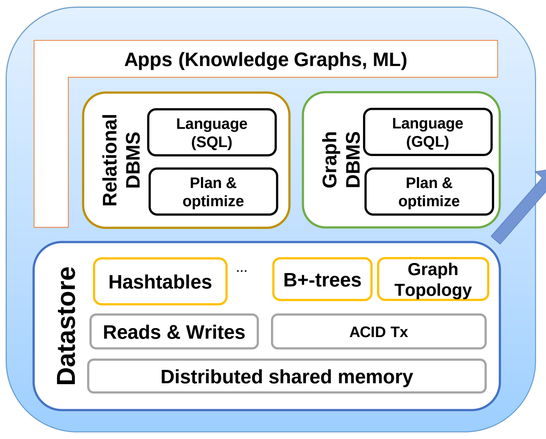
Dandelion: Hundreds of Millions of Distributed Replicated Transactions with Few Machines

Antonios Katsarakis Vasilis Gavrielatos Chris Jensen Nikos Ntarmos

Huawei Research

Datastores 101



Backbone of wide range of applications, services and systems (including DBMS)

Performance-critical intra-datacenter storage layer to access & manipulate data with

> *Efficiency*
high throughput, cost-effective

> *Programmability*
transactions, strong consistency

> *High availability*
replication and fault tolerance

Evolution of Datastores
(note: 10x leaps that need redesign)

Traditional Era

Traditional Datastores
Disk-based, slow network
➢ Efficiency: non-scalable, low performance

#1

NoSQL Datastores
Sacrifice programmability for performance
Ok for best-effort web-service but not critical app!
➢ Programmability: sacrifice transactions, consistency

#2
Exploit new HW for efficiency + programmability

New Era

Modern Datastores: e.g., Microsoft's FaRM [SOSP'15]
➢ Scalable, good perf., transactions, consistency, availability
- In-memory
- Fast (RDMA) network
- *FaRM* ➔ **140M TATP txs/sec (90 machines – 5TB)**

#3

Next-gen Datastores: academia + our goal
➢ Modern datastores + better perf., less cost, higher capacity
- HW acceleration: FPGA, ASICs, smart NICs/switches
- CXL expansion, Disaggregated memory, PMEM

More memory with less compute ➔ 50TB in 5-10 machines

Challenge: Can we achieve same or better performance with 10x fewer machines (compute) ?

Stoppers!

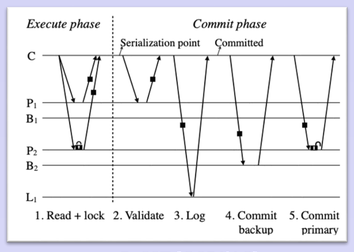
1. Fast **Networks**, but small packets ✗

***eRPC* [*NSDI'19 – best paper*]**
Efficient general-purpose RPCs
DPDK, UDP, RoCE/IB (two-sided) RDMA
Retransmissions
Support packets > MTU
- General purpose, Reliable, callback API
- Great for large/medium messages … **BUT**

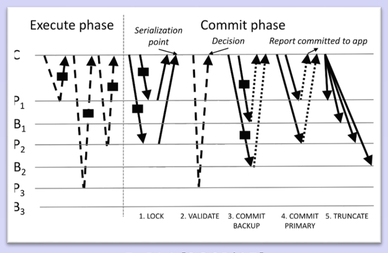
Issues with small (8-24B) requests
1. RoCE packets have 66B Header = 10-30% BW utilization
2. Too many packets, NIC/switch cannot keep up with packet rate

Datastores are dominated by
Small data messages → access/update small key-value pairs (e.g. 8-32B)
Tiny protocol logic messages → lock/unlock, commit/abort, truncate, etc.

2. Fault-tolerant strongly-consistent **Protocols** ✗
but complex and costly execution/commit phases
- lock-based execution phase or
- complex multiple round-trip (RTT) commit
- protocols have tradeoffs, but datastores no protocol alternatives

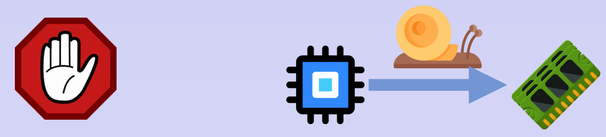


FaSST [OSDI'16]



FaRM [SOSP'15]

3. Concurrent in-memory **Data Structures** ✗
but practically blocking & not memory-aware



4. **CPUs** with many cores ✗
but cycles are wasted
on small packets, many protocol actions,
stalled waiting on memory, one-at-a-time request execution

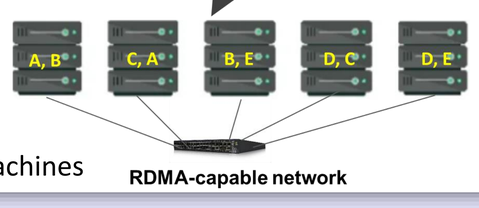
Dandelion: across-stack innovation + SW-HW co-design + batched execution (OLTP *like* OLAP)!

Dandelion

Context

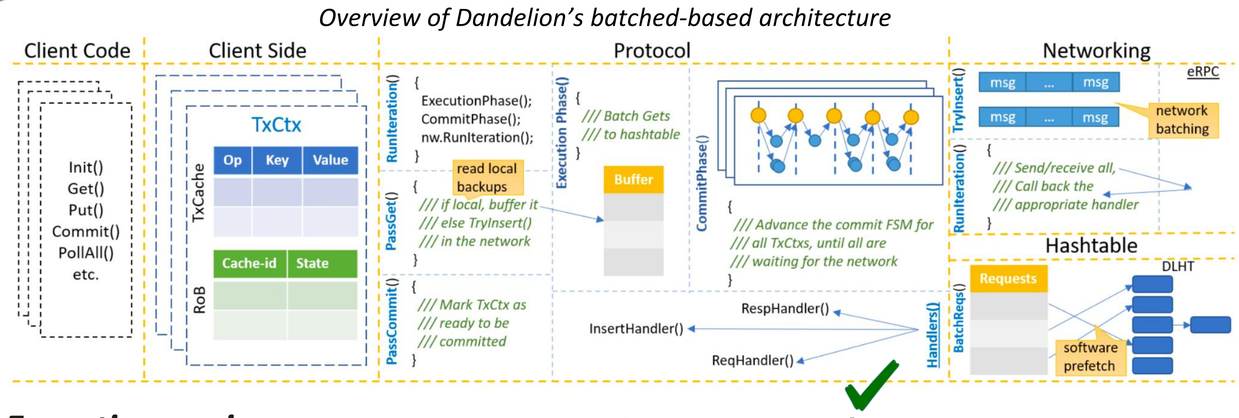
In-memory dataset: small/moderate sized **key-value** pairs
- **Distributed** across multiple machines for capacity
- **Replicated** for availability in the face of faults
- **Interactive Transactions**

```
tx.Init();
  auto val = tx.Get(A)
  if (val == 0) {
    tx.Put(B, 0);
  }
  tx.Insert(C, 1);
  tx.Delete (D)
tx.Commit();
```



Goal
1. **Reliable**: availability, strong consistency
2. **Programmable**: expressive, transactions
3. **Top performance/cost**:
e.g., better performance with 10x fewer machines

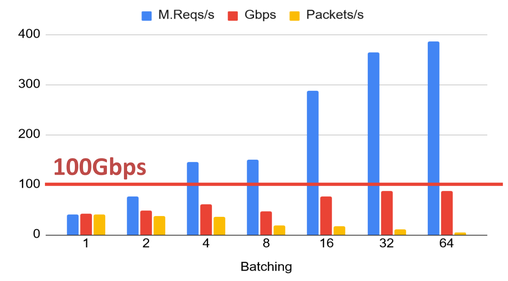
Overview of Dandelion's batched-based architecture



Execution engine: end-to-end **asynchronous & batched-based** → avoids wasting CPU cycles

Network

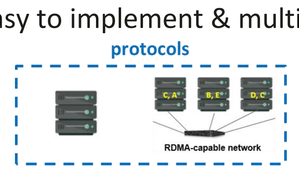
Datastore-optimized RPCs-over-RDMA ✓



Application-level batching
- necessary to fully utilize modern (100s Gbit net.)
- dandelion can batch 10s requests in one packet

Protocols

Easy to implement & multiplex protocols efficiently in Dandelion

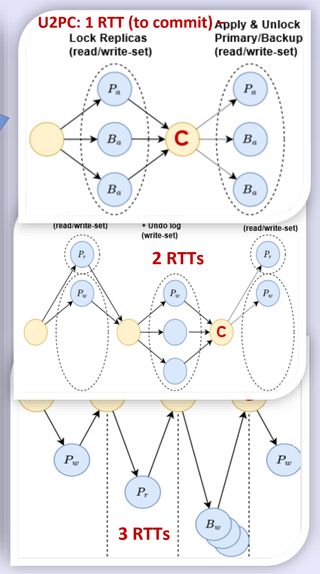


PaPoC'24
Unanimous 2PC: Fault-tolerant Distributed Transactions Can be Fast and Simple

3 Novel Fast Reliable Protocols: latency, throughput, lock-window
- Efficient logging and decentralized recovery
- Strongest consistency, Correctness: verified in TLA+
- High performance

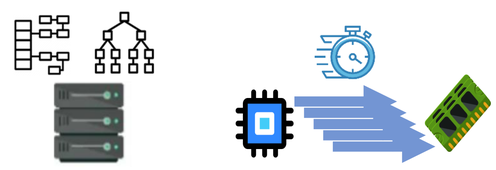
Lock-free execution phase (+ caching) ✓
- neither reads nor writes grab locks →
- programmability, performance, simple recovery

Fast reliable commit phase
- read-only / write-only txs: 0-1RTT
- read/write txs: 1RTT – 3RTTs (based on protocol) ✓



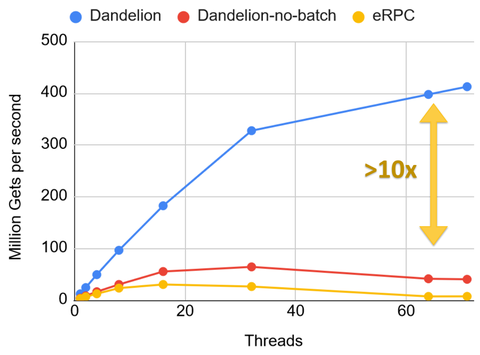
Data structures

Practically non-blocking and **memory-aware** ✓

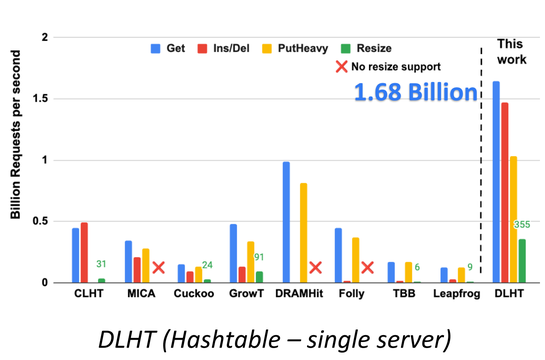


Dandelion Hashtable (DLHT) [Tree (DLTree)]
1. **Non-blocking requests [reads]**
2. **Most requests: 1 [few] memory access**
3. **Pipelined batched request processing**
→ overlap memory access + in-order completion
4. **Transaction-native and batched API** …

Networking

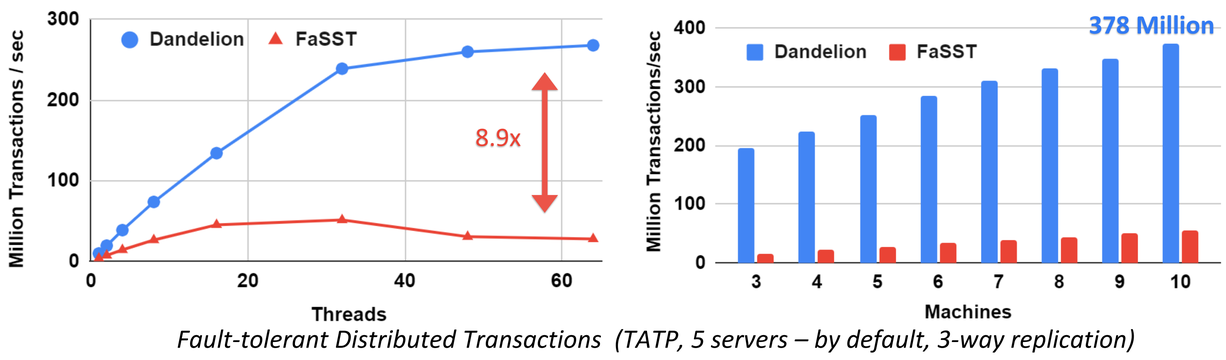


Data Structures



DLHT (Hashtable – single server)

Dandelion: network + protocols + data structures + async. batched engine



Fault-tolerant Distributed Transactions (TATP, 5 servers – by default, 3-way replication)

Dandelion networking **>10x** over eRPC

Dandelion Hashtable **>1.6B ops/sec** on a single server

Dandelion **>250M** distributed replicated txs (5 servers) | **12x-7x** faster than FaSST (3-10 servers)

Commodity servers: *18-core Intel Xeon Gold 6254 (2 sockets), 128GB DRAM* Network: *100Gbit RoCE*

HUAWEI